Loss-Conditional Physics-Informed Neural Networks: Amortising Across Parametric PDE Families

Anonymous Author(s)
Affiliation
Address
anon.email@example.com

Abstract

Physics-informed neural networks (PINNs) cast PDE solving as the minimisation of a composite residual / boundary / data loss whose weights w trade off the constraints. In practice the network must be retrained at every w a user wishes to inspect; adaptive weighting schemes (SA-PINN, ReLoBRaLo, Causal-PINN) sidestep this by committing the network to a single self-found weighting. We instead train one network conditioned on w, drawing fresh weights each step from a fixed prior; the construction is the loss-conditional networks of Dosovitskiy and Djolonga (2020) adapted to the residual-loss setting. The same architecture handles the case where w is replaced by a physical scalar parameter of the PDE. We prove that at a global minimum of the λ-expected objective the network is a per-λ residual minimiser for p_λ-almost every λ, and at test time K-shot averaging across λ matches the per-λ baselines. On viscous Burgers, parametric Helmholtz and Buckley–Leverett, one LC-PINN replaces an entire grid of retrained baselines: the amortisation factor against the strongest single-shot baseline is $\sim 29\times$ at $K = 100$, with training-cost break-even at $K \approx 4$.

1 Introduction

Physics-Informed Neural Networks (PINNs; Raissi et al., 2019) fit a single neural network to a single PDE by backpropagating the residual of the PDE itself. The composite loss combines a residual term, boundary and initial conditions, and (optionally) sparse measurements; the relative weights of these terms control the trained solution and are notoriously difficult to choose. Standard practice is grid search or self-adaptive scheduling (McClenny and Braga-Neto, 2020; Bischof and Kraus, 2021; Wang et al., 2024); both produce *one* solution for *one* weighting and must be repeated when the user wants to explore the tradeoff surface.

A separate stream of work in scientific machine learning attacks the same problem from the operator-learning angle: methods like FNO (Li et al., 2021) and DeepONet (Lu et al., 2021) amortise across an entire family of PDE instances by treating the right-hand side or the initial condition as a function-valued input. They produce a single network that responds to a continuum of inputs, but they require a training distribution of *paired* (input, solution) examples generated by an external solver; the PDE residual itself plays no role in training.

This paper sits between those two approaches. We adopt the PINN residual-loss machinery but replace the fixed weight vector by a *network input*, so a single PINN-style training run produces a continuum of solutions indexed by λ. The construction generalises the loss-conditional networks of Dosovitskiy and Djolonga (2020) — originally proposed for image-generation loss-mixture amortisation — to the parametric-PDE setting. The same architecture serves two distinct uses:

1. *Loss-weight amortisation* ($\lambda = w$): one network covers the full convex hull of admissible weight choices, removing the per-weight retraining cost and exposing the tradeoff surface as a function of λ.
2. *Parametric-coefficient amortisation* ($\lambda =$ a physical scalar parameter such as a Helmholtz wavenumber or a BL mobility ratio): one network covers the full range of that parameter, competing directly with operator-learning baselines while requiring no paired training data.

Contributions.

- We formulate the LC-PINN (Equations (2) and (3)) and prove a λ-invariance result: at a global optimum of the λ-expected loss, the conditional network is p_λ-a.e. a residual-minimiser of the parametric PDE indexed by λ (Proposition 1).
- On 1D Burgers (loss-weight amortisation) we benchmark LC-PINN against three adaptive-weight baselines (SA-PINN, ReLoBRaLo, Causal-PINN) and one operator-learning baseline (FNO) under a matched compute budget. LC-PINN matches the strongest single-shot baseline on rel-L^2 to within a factor of two and beats SA-PINN / ReLoBRaLo by an order of magnitude, breaking even on training compute after ~ 4 retrainings and amortising $\sim 29\times$ at $K = 100$ (Section 5.1).
- On 1D parametric Helmholtz (parametric-coefficient amortisation, $k \in [1, 10]$), one LC network covers the full k-range with rel-L^2 comparable to per-k retrained baselines (Section 5.3).
- On capillarity-corrected (viscous) Buckley–Leverett, LC-PINN is within a factor of two of the strongest single-shot baseline (ReLoBRaLo) while parameterising the entire loss-weight family with one trained network; we discuss the modelling assumption that constant-ε regularisation is a first-order proxy for saturation-dependent capillary diffusion (Section 5.2).
- We provide ablations for the K-shot inference budget, per-step λ-sample count, and uniform vs. log-uniform sampling distributions (Section 5.5); the sampling-distribution result is the empirical face of Remark 3 in Section 3.3.

2 Related work

Adaptive loss weighting in PINNs. The fragility of the composite-loss objective in the original PINN formulation (Raissi et al., 2019) has motivated a line of methods that adapt the residual / boundary / data weights during training. Self-adaptive PINN (SA-PINN) of McClenny and Braga-Neto (2020) attaches a trainable per-collocation-point weight that is updated by ascent on the residual, yielding an effective focusing of the network on hard-to-fit regions; their "points-with-weights" formulation is essentially a soft hard-example-mining for the residual. ReLoBRaLo (Bischof and Kraus, 2021) re-balances loss-term weights from running ratios of the absolute loss values, smoothed by an exponential moving average. Causal-PINN (Wang et al., 2024) weights the residual by a time-causal mask so the network does not collapse onto late-time dynamics before fitting the initial-time region. These all produce *one* solution for *one* (adaptively-found) weighting and would need to be retrained at every new weighting the user wishes to inspect.

Operator learning. A parallel line of work has built neural operators that map a function-valued input (boundary condition, forcing term, initial condition) to a function-valued solution. The Fourier Neural Operator (FNO) (Li et al., 2021) parameterises the operator in spectral space: linear layers on the lowest Fourier modes, plus a residual 1×1 convolution. DeepONet (Lu et al., 2021) factorises the operator into a branch network (encoding the input function at sensor points) and a trunk network (encoding the spatial coordinate), recombined by an inner product. Both methods amortise across a family of problems but require a precomputed dataset of (input, solution) pairs from a classical solver—they are supervised learners in function space. The PINN residual itself plays no role in their training. PI-DeepONet (Wang et al., 2021) re-introduces the residual but stays inside the operator-learning framing, treating the input function as the parameter rather than a scalar coefficient. Our LC-PINN sits between these two camps: it amortises like operator learning but uses the residual loss like a PINN.

Loss-conditional networks. Dosovitskiy and Djolonga (2020) introduced the loss-conditional networks idea in image generation: a single generator, conditioned on the loss weights of a multi-objective composite reconstruction loss, produces images that interpolate the tradeoff surface (sharpness vs. perceptual fidelity vs. adversarial realism). The network is trained by sampling the weight vector from a fixed prior at every step and applying the freshly-sampled weights to the per-instance reconstruction loss. LC-PINN takes the same conditional-on-weights idea and applies it to the PDE-residual setting; the formal λ-invariance result (Section 3.3) is, to our knowledge, the first analysis of the optimum-set structure of this construction in the residual-loss case.

Hypernetworks and meta-learning for PINNs. A separate amortisation strategy uses hypernetworks (Ha et al., 2017) or MAML-style meta-learning (Finn et al., 2017) to produce per-instance PINN weights. PI-MAML (Liu et al., 2022) demonstrates the meta-learning angle on parametric PDEs. These methods amortise at the level of network weights rather than a network input. They typically require a training distribution of full PDE *tasks* and a per-task inner-loop adaptation step at test time; LC-PINN trains a single network with no inner loop and produces predictions for every λ in one forward pass.

Extrapolation in λ vs. in physical parameters. The two amortisation modes we use—loss-weight amortisation and parametric-coefficient amortisation—place LC-PINN in different neighbourhoods of the literature. In the loss-weight mode, the network's role is closest to Dosovitskiy and Djolonga (2020). In the parametric-coefficient mode, the network's role is closest to FNO / DeepONet but parameterised over a low-dimensional scalar input ($d_\lambda = 1$ for Helmholtz wavenumber) rather than a function-valued input ($d_\lambda \sim$ grid resolution for operator learning). The two modes share an architecture; the choice of λ is task-driven.

3 Method

3.1 Loss-conditional formulation

A standard PINN (Raissi et al., 2019) approximates the solution of a single PDE $Fu = 0$ with auxiliary boundary, initial, and (optionally) data conditions $\{Bu = g_B, Iu = g_I, Du = g_D\}$ by minimising the weighted composite loss

$$\mathcal{L}^{\text{PINN}}(\theta;\, w) = w_F \left\|Fu_\theta\right\|_{L^2}^2 + w_B \left\|Bu_\theta - g_B\right\|_{L^2}^2 + w_I \left\|Iu_\theta - g_I\right\|_{L^2}^2 + w_D \left\|Du_\theta - g_D\right\|_{L^2}^2, \tag{1}$$

in the network parameters θ. The choice $w = (w_F, w_B, w_I, w_D)$ controls a strong tradeoff between satisfying the residual and matching the auxiliary constraints; in practice the weights are tuned by grid search or by an adaptive schedule (SA-PINN (McClenny and Braga-Neto, 2020), ReLoBRaLo (Bischof and Kraus, 2021), Causal-PINN (Wang et al., 2024)).

We replace the fixed-w problem with a *family* of problems indexed by a parameter vector $\lambda \in \Lambda$, where $\Lambda \subseteq \mathbb{R}^{d_\lambda}$ is either (i) the set of admissible loss weights itself ($\lambda = w$, $d_\lambda = 4$ for Burgers and BL), or (ii) a physical parameter of the PDE such as a wavenumber or mobility ratio ($\lambda = k$, $d_\lambda = 1$ for Helmholtz). The *loss-conditional* PINN (LC-PINN) is a single network

$$u_\theta : \Omega \times \Lambda \to \mathbb{R}, \qquad (x, \lambda) \mapsto u_\theta(x, \lambda), \tag{2}$$

trained to minimise the λ-expectation of the per-instance loss,

$$J(\theta) = \mathbb{E}_{\lambda \sim p_\lambda}\left[\mathcal{L}\big(\lambda;\, u_\theta(\cdot, \lambda)\big)\right], \tag{3}$$

where p_λ is a fixed sampling distribution (uniform on Λ unless stated otherwise). The architectural cost is one extra input of width d_λ at the first layer; the per-step compute is one forward/backward pass at a freshly drawn λ. This generalises the loss-conditional networks of Dosovitskiy and Djolonga (2020) from generative loss-mixture amortisation to the residual-loss setting in which the "loss mixture" is the PDE residual itself.

3.2 Inference: K-shot averaging across λ

At evaluation time we report a single rel-L^2 error per task. For the loss-weight conditioning case there is no canonical "true" weight, so following Dosovitskiy and Djolonga (2020) we average network predictions over K samples of λ drawn from p_λ:

$$\hat{u}(x) = \frac{1}{K} \sum_{i=1}^{K} u_\theta(x, \lambda^{(i)}), \qquad \lambda^{(i)} \overset{\text{i.i.d.}}{\sim} p_\lambda. \tag{4}$$

This K-shot averaging exposes the variance structure of the LC-PINN across the loss-weight family and underpins the amortisation analysis in Section 5: a baseline PINN must be retrained K times to produce K predictions; the LC-PINN produces all K from a single training run.

3.3 λ-invariance at the LC optimum

The empirical claim of this paper is that a single LC-PINN $u_\theta(x, \lambda)$, trained with λ sampled from a probability measure p_λ, parameterises the parametric PDE family $\{F_\lambda u = 0\}_{\lambda \in \Lambda}$ *simultaneously* in λ. The following proposition makes precise the sense in which this holds at a global optimum of the LC-PINN objective.

Setup. Let $\Lambda \subseteq \mathbb{R}^{d_\lambda}$ be the parameter set, p_λ a probability measure on Λ with full support, and $\Omega \subseteq \mathbb{R}^{d_x}$ the spatial–temporal domain with sampling measure p_Ω. Each $\lambda \in \Lambda$ indexes a residual operator F_λ together with associated boundary, initial, and (optional) data-fit operators $\{B_\lambda, I_\lambda, D_\lambda\}$. For a candidate field $v : \Omega \to \mathbb{R}$, write the per-$\lambda$ functional

$$\mathcal{L}(\lambda; v) \;=\; w_F \, \|F_\lambda v\|^2_{L^2(\Omega, p_\Omega)} + w_B \, \|B_\lambda v\|^2_{L^2(\partial\Omega)} + w_I \, \|I_\lambda v\|^2_{L^2(t=0)} + w_D \, \|D_\lambda v\|^2_{L^2},$$

with strictly positive loss weights (w_F, w_B, w_I, w_D). The LC-PINN training objective is the p_λ-expectation

$$J(\theta) \;=\; \int_\Lambda \mathcal{L}\big(\lambda;\, u_\theta(\,\cdot\,, \lambda)\big)\, dp_\lambda(\lambda).$$

Assume:

(A1) the network class $\{u_\theta(\cdot, \lambda) : \theta \in \Theta\}$ contains, for every λ, a residual-minimiser $u^\star_\lambda \in \arg\min_v \mathcal{L}(\lambda; v)$ with $\mathcal{L}(\lambda; u^\star_\lambda) = 0$;

(A2) the map $\lambda \mapsto \mathcal{L}(\lambda; u_\theta(\cdot, \lambda))$ is dp_λ-integrable for every θ;

(A3) u_θ is jointly continuous in (x, λ).

Proposition 1 (Residual-minimiser at the LC optimum)**.** *Let $\theta^\star \in \arg\min_\theta J(\theta)$. Under* (A1)–(A3)*, for p_λ-almost every $\lambda \in \Lambda$ the section $u_{\theta^\star}(\cdot, \lambda)$ is a residual-minimiser of the parametric PDE indexed by λ:*

$$\mathcal{L}\big(\lambda;\, u_{\theta^\star}(\,\cdot\,, \lambda)\big) \;=\; 0 \qquad \textit{for } p_\lambda\textit{-a.e. } \lambda \in \Lambda.$$

In particular, on $\mathrm{supp}(p_\lambda)$ *the LC-PINN section satisfies $F_\lambda u_{\theta^\star}(\cdot, \lambda) = 0$ in $L^2(\Omega, p_\Omega)$ and the auxiliary constraints to the same precision.*

Sketch proof. The integrand $\lambda \mapsto \mathcal{L}(\lambda; u_\theta(\cdot, \lambda))$ is non-negative for every θ and every λ, since each summand is a squared L^2-norm. By (A1) the per-λ minimum is zero, so $\inf_\theta J(\theta) \geq 0$ and the lower bound is attainable: any measurable selector $\lambda \mapsto \theta_\lambda$ with $u_{\theta_\lambda}(\cdot, \lambda) = u^\star_\lambda$ exists pointwise under (A1), and a single $\theta^\star$ achieving $J(\theta^\star) = 0$ exists when the network class is rich enough to parameterise the family $\{u^\star_\lambda\}_{\lambda \in \Lambda}$ jointly (e.g. universal approximation in $C(\Omega \times \Lambda)$). For such $\theta^\star$,

$$0 \;=\; J(\theta^\star) \;=\; \int_\Lambda \mathcal{L}\big(\lambda;\, u_{\theta^\star}(\,\cdot\,, \lambda)\big)\, dp_\lambda(\lambda),$$

and the integrand is non-negative; the standard "vanishing-integral of a non-negative function" lemma then gives $\mathcal{L}(\lambda; u_{\theta^\star}(\cdot, \lambda)) = 0$ for p_λ-a.e. λ. □

Remark 1 (where the support assumption matters). The conclusion is a p_λ-a.e. statement on Λ: nothing prevents the LC-PINN from being arbitrarily wrong on a p_λ-null set, including the boundary of Λ if p_λ has no mass there. In our experiments $p_\lambda = \mathrm{Uniform}(\Lambda)$ has full support, so the p_λ-a.e. statement coincides with Lebesgue-a.e. on Λ.

Remark 2 (finite-capacity gap). In practice (A1) is replaced by the network having *ε-residual capacity* for the family: there exists θ with $\mathcal{L}(\lambda; u_\theta(\cdot, \lambda)) \leq \varepsilon$ for p_λ-a.e. λ. The same argument then yields $\mathcal{L}(\lambda; u_{\theta^\star}(\cdot, \lambda)) \leq \varepsilon$ p_λ-a.e., which is the practically observable form of the result and matches the per-λ rel-L^2 tables in Section 5.

Remark 3 (sampling-distribution invariance). If p_λ and $\tilde{p}_\lambda$ are two probability measures with the same support $\Lambda_0 \subseteq \Lambda$, the sets of *a.e.-residual-minimisers* on Λ_0 coincide. So the optimum-set of the LC objective on $\mathrm{supp}(p_\lambda)$ does not depend on the precise sampling law—only on the support. This is an asymptotic statement; at finite training budget the law of p_λ governs which slices receive enough gradient signal, which we observe empirically (Section 5.5).

3.4 Network and training details

All experiments use a fully-connected tanh network with hidden widths $[64, 64, 64, 64]$, Xavier initialisation, and a single real-valued output head; the conditioning vector λ is concatenated to the spatial coordinates at the input layer. Training uses Adam ($\eta = 10^{-3}$) with cosine annealing to zero over the full training horizon and gradient-norm clipping at 1.0. Each step samples $\lambda \sim p_\lambda$ and evaluates the per-instance loss at the current batch of collocation points; in the parametric-coefficient setting, λ is mapped to a normalised range $\lambda_{\mathrm{norm}} \in [-1, 1]$ before being fed to the network. Implementation: PyTorch 2.9 on Apple M4 Max via the MPS backend. Full hyperparameters and per-equation collocation budgets are listed in Appendix A.

4 Experiments

We evaluate LC-PINN on three parametric PDE families that span the two amortisation modes of Section 3: a loss-weight family on viscous Burgers and Buckley–Leverett ($\lambda = w$, $d_\lambda = 4$) and a parametric-coefficient family on Helmholtz ($\lambda = k$, the wavenumber). We compare against two adaptive-loss-weighting PINN baselines (SA-PINN (McClenny and Braga-Neto, 2020), ReLoBRaLo (Bischof and Kraus, 2021)), one time-causal PINN baseline (Causal-PINN (Wang et al., 2024), applicable to Burgers / BL only as Helmholtz is elliptic), and one operator-learning baseline (FNO (Li et al., 2021)).

Test problems and references. *Burgers*: $u_t + uu_x - \nu u_{xx} = 0$ with $\nu = 0.01/\pi$ and $u(0, x) = -\sin(\pi x)$ on $[-1, 1] \times [0, 1]$, periodic in x. Reference computed by a Fourier-spectral / Radau IVP solver to rel-residual $< 10^{-6}$. *Buckley–Leverett*: $s_t + f'(s)\, s_x - \varepsilon s_{xx} = 0$ with the standard fractional flow $f(s) = s^2/(s^2 + M(1-s)^2)$, $M \in [1/3, 3]$, viscous regularisation $\varepsilon = 10^{-2}$, Riemann initial data $s_L = 1$, $s_R = 0$. Reference computed by Lax–Friedrichs flux on $n_x = 1000$ with centred finite-difference diffusion. *Helmholtz*: $u_{xx} + k^2 u = f(x; k)$ on $[0, 1]$ with $k \in [1, 10]$, manufactured target $u(x; k) = \sin(\pi x)\cos(kx)$ (forcing f chosen so this is the solution), homogeneous Dirichlet boundary conditions.

Training budgets. All PINN-family methods (LC, SA, ReLoBRaLo, Causal) train to a fixed Adam-step budget; LC and the per-λ baselines share the same per-step compute. Burgers: 50k Adam + 5k L-BFGS outer iterations. Buckley–Leverett: 50k Adam. Helmholtz: 50k Adam (no L-BFGS in the headline configuration to keep the λ-grid-vs-LC comparison clean). Hidden widths $[64, 64, 64, 64]$ throughout. FNO: 2000 epochs at width 64, modes 64, 4 spectral blocks, on a 512-pair operator-learning training set.

Evaluation. For LC-PINN in loss-weight mode (Burgers, BL) we report the K-shot averaged rel-L^2 at $K = 100$ unless stated otherwise; the K-eval ablation in Section 5.5 sweeps $K \in \{25, 50, 100, 200, 400\}$. In parametric-coefficient mode (Helmholtz) we report rel-L^2 at a fixed five-point grid of k values, averaged across the grid and then over seeds:

$$k \in \{1.00,\ 3.25,\ 5.50,\ 7.75,\ 10.00\}.$$

All numbers are mean ± std over 4 random seeds. Wall times are measured on a single Apple M4 Max via the MPS backend.

Baselines. *SA-PINN*: per-collocation-point trainable weight, ascent on the residual; Adam phase 10k, L-BFGS phase 5k iters with weights frozen. *ReLoBRaLo*: 3-term loss-weight balancer with softmax-on-loss-ratio rebalancing, smoothed by EMA; $\alpha = 0.999$, $\tau = 0.1$, $\rho_{\mathrm{mean}} = 0.999$. *Causal-PINN*: time-causal residual mask $w(t) = \exp(-\varepsilon\, R(<t))$ with $\varepsilon = 100$ (Burgers, BL only). *FNO*: width 64, modes 64, 4 SpectralConv1d

+ Conv1d-shortcut blocks; Adam ($\eta = 10^{-3}$, weight decay 10^{-4}), StepLR with step = epochs/5 and $\gamma = 0.5$; per-sample rel-L^2 training loss (Li et al., 2021). The operator-learning training set is 512 random Fourier ICs (truncated 6-mode spectrum, decaying amplitude) solved by the same Radau scheme; the test IC is the canonical $-\sin(\pi x)$, in-distribution by construction.

5 Results

We organise the results around the two amortisation modes. The loss-weight mode (Burgers, BL) is reported with K-shot averaged rel-L^2; the parametric-coefficient mode (Helmholtz) is reported as the rel-L^2 averaged across the five-point k-grid. All numbers are mean ± std over 4 random seeds.

5.1 Burgers, loss-weight mode

Table 1 reports rel-L^2 at the canonical Burgers test problem ($\nu = 0.01/\pi$, $u(0, x) = -\sin(\pi x)$, $t = 1$). The headline comparison is between LC-PINN ($K = 100$ shots from one trained network) and the per-λ baselines, each trained to its own self-found weighting.

Table 1: Burgers, rel-L^2 at the test IC $-\sin(\pi x)$. LC-PINN reports $K = 100$-shot averaging; baselines are single-shot at their self-found weighting (or operator-learned in the FNO case).

Method	rel-L^2 (mean ± std)	wall time / seed
LC-PINN ($K = 100$)	$3.47 \times 10^{-3} \pm 2.73 \times 10^{-3}$	24.8 min
Causal-PINN	$2.21 \times 10^{-3} \pm 6.72 \times 10^{-4}$	7.1 min
SA-PINN	$1.68 \times 10^{-1} \pm 3.87 \times 10^{-2}$	4.6 min
ReLoBRaLo	$1.82 \times 10^{-1} \pm 1.71 \times 10^{-2}$	5.4 min
FNO (operator)	$5.37 \times 10^{-1} \pm 1.89 \times 10^{-1}$	8.0 min

LC-PINN matches Causal-PINN (the strongest single-shot baseline) on rel-L^2 to within a factor of two, and beats SA-PINN and ReLoBRaLo by an order of magnitude. The LC-PINN figure is the cost of *one* training run producing predictions for any λ in the family; each baseline figure is the cost of one training run producing predictions at *one* weighting. The relevant amortisation factor is therefore $A(K) = K \cdot T_{\text{base}}/T_{\text{LC}}$, the ratio of K separate baseline retrainings to one LC training run. Break-even ($A(K) = 1$) is at $K^\star = T_{\text{LC}}/T_{\text{base}} \approx 3.5$ retrainings against Causal-PINN, 4.6 against ReLoBRaLo, and 5.4 against SA-PINN; at $K = 100$ the factor is $\sim 29\times$, $\sim 22\times$, and $\sim 19\times$ respectively. LC trades a small accuracy loss against Causal-PINN at the canonical IC for coverage of the entire λ-family from a single training run.

5.2 Buckley–Leverett, loss-weight mode

Table 2 reports rel-L^2 on the viscous-regularised BL problem ($\varepsilon = 10^{-2}$). At the original zero-viscosity formulation all PINN-family methods saturate at rel-$L^2 \sim 0.5$ because of the sharp shock; the viscous regularisation is a standard physics-side modification that the methods can resolve.

Table 2: Viscous-regularised BL ($\varepsilon = 10^{-2}$), rel-L^2 at the test snapshots. LC-PINN reports $K = 25$-shot averaging over loss-weight λ; baselines are single-shot at their self-found weighting.

Method	rel-L^2 (mean ± std)
LC-PINN ($K = 25$)	$1.03 \times 10^{-2} \pm 1.30 \times 10^{-3}$
SA-PINN	$4.60 \times 10^{-1} \pm 6.74 \times 10^{-2}$
ReLoBRaLo	$5.09 \times 10^{-3} \pm 7.79 \times 10^{-4}$

On viscous BL, the strongest single-shot baseline (ReLoBRaLo, 5.09×10^{-3}) edges LC-PINN (1.03×10^{-2}) by a factor of ~ 2 at single-instance comparison, while SA-PINN fails to converge below ~ 0.5. The per-λ

accuracy advantage of ReLoBRaLo is real but narrowly defined: the reported number is the cost of one training run at *one* self-found weighting, and a user wishing to inspect a different weighting would pay that cost again. LC-PINN, in contrast, parameterises the entire loss-weight simplex from one training run.

5.3 Helmholtz, parametric-coefficient mode

Table 3 reports rel-L^2 averaged across the five-point k-grid. Here LC-PINN's input is the wavenumber k itself; the baselines are retrained at each k_{train}.

Table 3: Helmholtz, rel-L^2 averaged across $k \in \{1.00, 3.25, 5.50, 7.75, 10.00\}$.

Method	rel-L^2 (mean $\pm$ std)	wall time
LC-PINN (one network)	$9.81 \times 10^{-3} \pm 2.47 \times 10^{-3}$	39.0 min total
SA-PINN (per-k, 5 retrainings)	$3.23 \times 10^{-3} \pm 2.90 \times 10^{-3}$	11.4 min/k
ReLoBRaLo (per-k, 5 retrainings)	$3.87 \times 10^{-3} \pm 2.82 \times 10^{-3}$	7.9 min/k

A single LC-PINN training run produces a network that, given any $k \in [1, 10]$, returns the manufactured solution at rel-$L^2 \sim 10^{-2}$. The baselines must be retrained at every k of interest, so the amortisation factor is proportional to the size of the k-grid the user wishes to evaluate.

5.4 Operator-learning baseline (FNO)

We trained a 1D FNO (width 64, modes 64, 4 spectral blocks, $\sim$1.07M parameters) on 512 random Fourier ICs at the same viscosity $\nu = 0.01/\pi$, predicting the final-time snapshot $u(x, 1)$. Inference on the canonical test IC $-\sin(\pi x)$ gives rel-$L^2 = 5.37 \times 10^{-1} \pm 1.89 \times 10^{-1}$ across 4 seeds (Table 1, last row). The high error is consistent with the difficulty of operator-learning near-shock-forming Burgers from a 512-pair dataset; reference FNO benchmarks use larger viscosity ($\nu \sim 0.1$, no shock) where the operator is smoother. We do not interpret this as a deficiency of FNO: it is solving a different, harder problem — the operator $u(0, \cdot) \mapsto u(1, \cdot)$ over a function-space distribution of ICs — and would close the gap with $\nu = 0.1$ data or roughly an order of magnitude more training pairs. The point is that PINN-style methods (LC-PINN, Causal-PINN) reach $\sim 10^{-3}$ on the canonical IC *without paired training data* at all.

5.5 Ablations

K-shot averaging. Sweeping $K \in \{25, 50, 100, 200, 400\}$ on the trained LC-PINN gives essentially constant rel-L^2 (3.51×10^{-3} at $K = 25$, 3.49×10^{-3} at $K = 100$, 3.49×10^{-3} at $K = 400$; spread $\pm 2.7 \times 10^{-3}$ stays roughly constant). This is consistent with the λ-invariance proposition: at the optimum each λ-slice is already a residual minimiser, so increasing K adds samples but no new information.

n_λ per training step. At a matched 25k-epoch budget on Burgers (2 seeds), sweeping $n_\lambda \in \{1, 4, 16\}$ per step gives non-monotone behaviour: $n_\lambda = 1 \to 1.69 \times 10^{-2}$, $n_\lambda = 4 \to 1.22 \times 10^{-2}$, $n_\lambda = 16 \to 1.50 \times 10^{-1}$. The sweet spot is around $n_\lambda \sim 4$: too few λ per step makes the gradient estimate of the p_λ-expectation noisy; too many concentrates compute into fewer optimisation steps and the network does not converge in the fixed budget. At the headline 50k-epoch budget we use $n_\lambda = 4$.

Sampling distribution p_λ (uniform vs. log-uniform). At the matched 25k-epoch / $n_\lambda = 4$ / 2-seed budget on Burgers, swapping p_λ from uniform on the simplex to log-uniform gives rel-L^2 of $1.22 \times 10^{-2} \pm 5.14 \times 10^{-3}$ (uniform) versus $1.74 \times 10^{-1} \pm 9.26 \times 10^{-3}$ (log-uniform), a $\sim 14\times$ degradation. We read this not as a contradiction with Remark 3 (an asymptotic-optimum statement) but as a finite-budget effect: log-uniform places dominant mass on small loss weights, weakening the gradient signal at large λ values where the residual contributes most to the test-relevant slices. The optima coincide *at convergence* (the supports of both priors are equal); training-budget-limited LC-PINNs feel the difference in coverage. We use uniform sampling at the headline configuration.

6 Discussion

When LC-PINN amortises and when it does not. The empirical picture across our three PDE families is that LC-PINN's advantage is monotone in the cost of a baseline solve. On Burgers (loss-weight family $d_\lambda = 4$, single-shot Adam+L-BFGS expensive) the amortisation factor against the strongest baseline at $K = 100$ is $\sim 29\times$, with training-cost break-even at $K \approx 4$ retrainings. On Helmholtz, where $\lambda = k$ is a genuinely physical parameter and each baseline retraining is one full PINN, the factor collapses to a single ratio between the LC training time and the cost of a k-grid (Section 5.3). On Buckley–Leverett the gain is modest because each per-λ baseline is itself cheap. We read this as evidence that the amortisation metric is not a property of the method alone but of the family $(\Lambda, p_\lambda, \mathcal{L})$ being amortised.

Comparison to operator learning. FNO and DeepONet amortise on a different axis (a function space of inputs vs. a scalar parameter λ) and require precomputed solver pairs at training time. On the canonical Burgers benchmark $\nu = 0.01/\pi$, $u(0, x) = -\sin(\pi x)$, a near-shock develops at $t = 1$ that lies at the edge of FNO's modal bandwidth, and the operator-learning baseline does not match LC-PINN at the test IC. We do not interpret this as "FNO is worse": it is solving a different, harder problem (the operator $u(0, \cdot) \mapsto u(1, \cdot)$, generalising over a function-space distribution of ICs). The two methods are complementary — when one has a precomputed dataset of (input, solution) pairs, FNO is the right tool; when one does not, but does have the PDE residual and a parametric coefficient, LC-PINN is.

The λ-invariance theorem in practice. Proposition 1 states that at a global minimum of J, the network is a per-λ residual minimiser for p_λ-almost every λ. Two practical consequences that we observed: (i) the K-shot averaged prediction is empirically stable across K for $K \gtrsim 25$ (Section 5.5, K-eval ablation), consistent with each λ-slice having already converged; and (ii) although the asymptotic optimum-set is invariant to the precise sampling law (Remark 3 in Section 3.3), at finite training budget the law of p_λ matters for which slices receive enough gradient signal: log-uniform sampling on Burgers degrades rel-L^2 by a factor of ~ 14 relative to uniform at the matched 25k-epoch budget (Section 5.5). Remark 3 is therefore an asymptotic statement; finite-budget LC-PINN training is sensitive to p_λ through coverage rather than through the optimum-set.

Limitations. *Capacity*: a single network must fit the entire λ-family; on regions of Λ where solutions vary sharply (e.g. shock formation in BL across mobility ratios), the per-λ accuracy can lag the equal-budget single-λ PINN. *No solver data*: by construction LC-PINN cannot exploit precomputed high-fidelity data the way operator learners can; the comparison to FNO is therefore against a methodology that consumes a different kind of supervision. *Sampling distribution*: the prior p_λ governs which slices of the family the network learns well; for tail-heavy λ-priors the corresponding tail-slice predictions degrade. *Loss landscape*: the λ-conditioning interacts with the multi-objective tradeoffs in the residual and BCs, and we do not have a sharp theoretical characterisation of which λ-priors are well-conditioned for a given PDE family.

Future work. Three directions are immediate. First, a principled prior over the λ-distribution: at present we use uniform or log-uniform, but the right prior for, say, a viscosity coefficient that varies across many decades is itself a question. Second, the spectral case: LC-PINN can be applied to eigenproblems via the Ky Fan variational form, where the loss-conditional structure becomes a conditioning on mode index; this opens an entirely different family of problems. Third, combining the LC trick with operator learning: condition an FNO on λ to obtain an operator that amortises both over the function space of inputs *and* over a scalar physical parameter.

References

R. Bischof and M. A. Kraus. Multi-objective loss balancing for physics-informed deep learning. *arXiv preprint arXiv:2110.09813*, 2021.

A. Dosovitskiy and J. Djolonga. You only train once: loss-conditional training of deep networks. In *International Conference on Learning Representations (ICLR)*, 2020.

C. Finn, P. Abbeel, and S. Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International Conference on Machine Learning (ICML)*, 2017.

D. Ha, A. Dai, and Q. V. Le. Hypernetworks. In *International Conference on Learning Representations (ICLR)*, 2017.

Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

X. Liu, X. Zhang, W. Peng, W. Zhou, and W. Yao. A novel meta-learning initialization method for physics-informed neural networks. *Neural Computing and Applications*, 34:14511–14534, 2022.

L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.

L. McClenny and U. Braga-Neto. Self-adaptive physics-informed neural networks using a soft attention mechanism. *arXiv preprint arXiv:2009.04544*, 2020.

M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

S. Wang, H. Wang, and P. Perdikaris. Learning the solution operator of parametric partial differential equations with physics-informed deeponets. *Science Advances*, 7(40):eabi8605, 2021.

S. Wang, S. Sankaran, and P. Perdikaris. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 421:116813, 2024.

A Hyperparameters and training details

This appendix collects the hyperparameters used for every method in Section 4. All numbers are the configuration that generated the headline tables in Section 5.

Shared LC-PINN configuration. Network: 4-layer MLP, hidden widths $[64, 64, 64, 64]$, tanh activations, Xavier initialisation, λ concatenated to the spatial–temporal input. Optimiser: Adam, $\eta = 10^{-3}$, cosine-annealed to 10^{-5} over the full budget, gradient clip 1.0. Mini-batch composition: $n_\lambda = 4$ parameter samples per step, each with the per-equation collocation counts below. Backend: PyTorch 2.9 with the MPS device on a single Apple M4 Max.

Per-equation collocation and budgets. Table 4 lists the LC-PINN collocation counts and step budgets per PDE family.

Table 4: LC-PINN collocation counts and step budgets per PDE family.

PDE	interior	boundary	initial	Adam / L-BFGS
Burgers	1024	200	200	50k / 5k
BL (viscous)	1024	200	200	50k / 0
Helmholtz	1024	100	—	50k / 0

Baseline configurations. All single-λ baselines share the LC-PINN backbone (same widths, activation, initialisation, and Adam schedule) so that the amortisation comparison is on equal architectural footing.

- *SA-PINN (McClenny and Braga-Neto, 2020)*: per-collocation-point trainable weight ascended on the residual; Adam phase 10k iterations, then weights frozen and L-BFGS phase 5k iterations on the unweighted loss.
- *ReLoBRaLo (Bischof and Kraus, 2021)*: 3-term loss-weight balancer (residual / boundary / initial) with softmax over log-loss-ratios, smoothed by EMA. Hyperparameters $\alpha = 0.999$, $\tau = 0.1$, $\rho_{\text{mean}} = 0.999$.
- *Causal-PINN (Wang et al., 2024)*: time-causal residual mask $w(t) = \exp(-\varepsilon\, R(<t))$ with $\varepsilon = 100$. Burgers and BL only.
- *FNO (Li et al., 2021)*: width 64, 64 Fourier modes, 4 `SpectralConv1d` + `Conv1d`-shortcut blocks (~ 1.07M parameters). Optimiser: Adam, $\eta = 10^{-3}$, weight decay 10^{-4}, `StepLR` with step = epochs/5 and $\gamma = 0.5$. Loss: per-sample rel-L^2. Training set: 512 random Fourier initial conditions (truncated 6-mode spectrum, decaying amplitude) solved by the same Radau scheme used for the reference; 2000 epochs.

Reference solutions. Burgers reference: Fourier-spectral spatial discretisation with a Radau IIA implicit time integrator, integrated to relative residual $< 10^{-6}$ on a 512×1001 space–time grid. Buckley–Leverett reference: explicit Lax–Friedrichs flux on $n_x = 1000$ with centred-difference viscous regularisation, CFL-limited time stepping. Helmholtz reference: closed-form manufactured solution $u(x; k) = \sin(\pi x)\cos(kx)$ with the forcing f chosen to match.

Evaluation grids. Burgers / BL test errors are computed on a 256×101 space–time grid; Helmholtz on a 1024-point spatial grid. The K-shot averaged LC-PINN prediction is the mean of K forward passes at independent λ samples from p_λ; we use $K = 100$ for Burgers and $K = 25$ for BL in the headline tables.

Compute. Each run uses a single Apple M4 Max with the PyTorch MPS backend. Reported wall times are end-to-end per seed (model construction, training, and evaluation), measured with `time.perf_counter`.

Reproducibility. Random seeds: $\{0, 1, 2, 3\}$ for the four headline replicates; $\{0, 1\}$ for the 2-seed ablation budgets. Each seed controls the PyTorch global RNG and the NumPy RNG used to draw collocation points and λ-samples. The full training script, equation modules, and reference solvers are released with the camera-ready version.